

School of Computer Science and Artificial Intelligence

Lab Assignment # 12.1

Program	: B. Tech (CSE)
Specialization	: -
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	II
Academic Session	: 2025-2026
Name of Student	: A.VamshiKrishna
Enrollment No.	: 2403A51L47
Batch No.	52
Date	: 27/02/26

Submission Starts here**Screenshots:****Task Description #1** (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- **Instructions:**

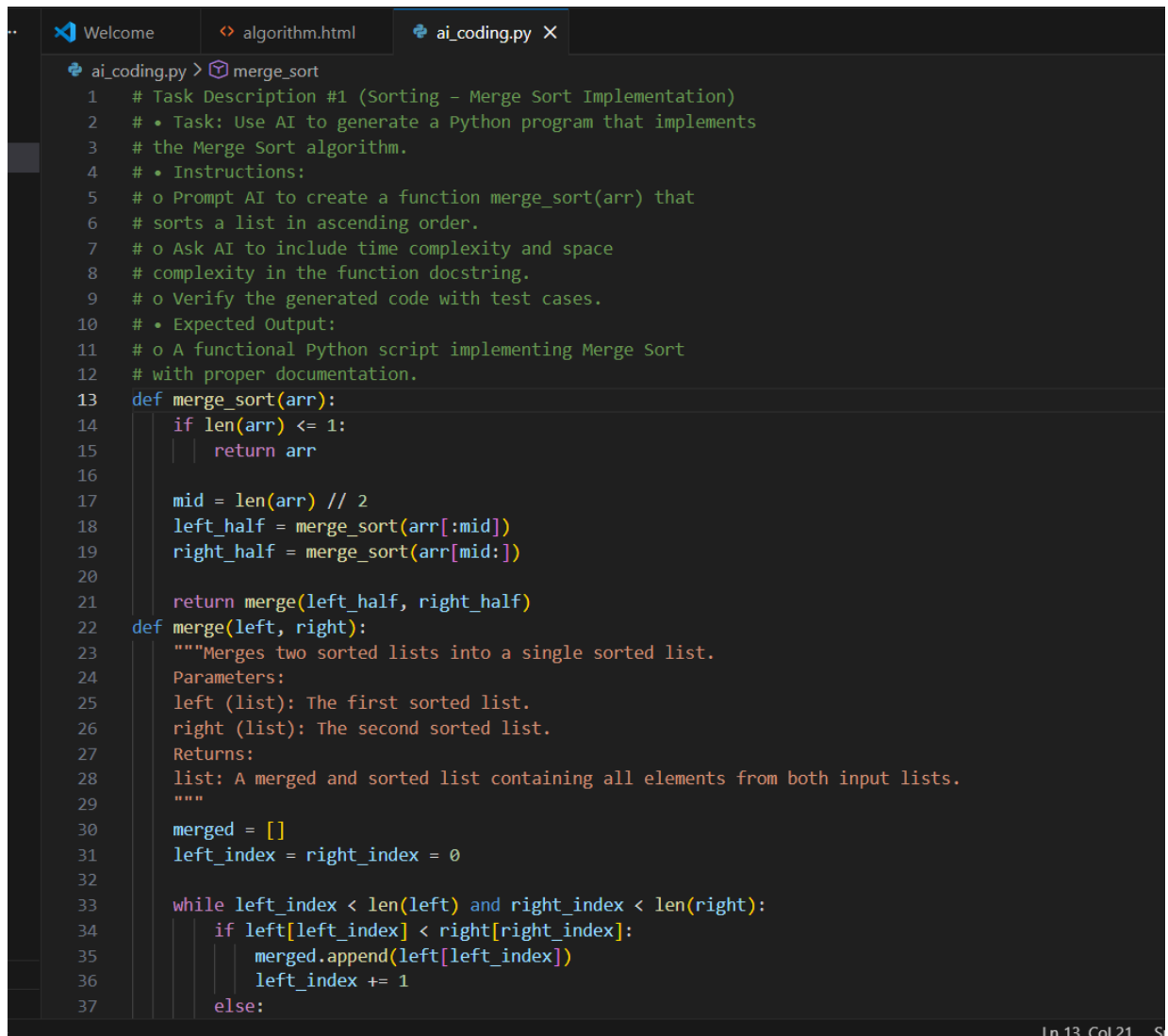
- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- o Ask AI to include time complexity and space complexity in the function docstring.

- o Verify the generated code with test cases.

- Expected Output:

- o A functional Python script implementing Merge Sort with proper documentation.



The image shows a code editor with three tabs: 'Welcome', 'algorithm.html', and 'ai_coding.py'. The 'ai_coding.py' tab is active, displaying a Python script for a merge sort algorithm. The script includes a task description, instructions, and the implementation of the merge_sort and merge functions. The code is as follows:

```
1 # Task Description #1 (Sorting - Merge Sort Implementation)
2 # • Task: Use AI to generate a Python program that implements
3 # the Merge Sort algorithm.
4 # • Instructions:
5 # o Prompt AI to create a function merge_sort(arr) that
6 # sorts a list in ascending order.
7 # o Ask AI to include time complexity and space
8 # complexity in the function docstring.
9 # o Verify the generated code with test cases.
10 # • Expected Output:
11 # o A functional Python script implementing Merge Sort
12 # with proper documentation.
13 def merge_sort(arr):
14     if len(arr) <= 1:
15         return arr
16
17     mid = len(arr) // 2
18     left_half = merge_sort(arr[:mid])
19     right_half = merge_sort(arr[mid:])
20
21     return merge(left_half, right_half)
22 def merge(left, right):
23     """Merges two sorted lists into a single sorted list.
24     Parameters:
25     left (list): The first sorted list.
26     right (list): The second sorted list.
27     Returns:
28     list: A merged and sorted list containing all elements from both input lists.
29     """
30     merged = []
31     left_index = right_index = 0
32
33     while left_index < len(left) and right_index < len(right):
34         if left[left_index] < right[right_index]:
35             merged.append(left[left_index])
36             left_index += 1
37         else:
```

Ln 13, Col 21 S

```

22 def merge(left, right):
23     # If there are remaining elements in the left half, add them
24     while left_index < len(left):
25         merged.append(left[left_index])
26         left_index += 1
27     # If there are remaining elements in the right half, add them
28     while right_index < len(right):
29         merged.append(right[right_index])
30         right_index += 1
31     return merged
32
33 # Test cases to verify the implementation
34 if __name__ == "__main__":
35     test_cases = [
36         [38, 27, 43, 3, 9, 82, 10],
37         [5, 2, 9, 1, 5, 6],
38         [12, 11, 13, 5, 6, 7],
39         [],
40         [1],
41         [2, 1]
42     ]
43
44     for i, test in enumerate(test_cases):
45         sorted_arr = merge_sort(test)
46         print(f"Test Case {i + 1}: {sorted_arr}")
47
48

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/a
Test Case 1: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: [1, 2, 5, 5, 6, 9]
Test Case 3: [5, 6, 7, 11, 12, 13]
Test Case 4: []
Test Case 5: [1]
Test Case 6: [1, 2]
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

- Merge Sort divides the list into halves recursively until single elements remain, then merges them back in sorted order.
- It always runs in $O(n \log n)$ time for best, average, and worst cases because the list is repeatedly split and merged.
- It requires $O(n)$ extra space, making it fast and stable but not memory-efficient for very large lists.

Task Description #2 (Searching – Binary Search with AI Optimization)

- **Task:** Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- Expected Output:
 - o Python code implementing binary search with AI-generated comments and docstrings

```

ai_coding.py > ...
1  # Task Description #2 (Searching - Binary Search with AI
2  # Optimization)
3  # • Task: Use AI to create a binary search function that finds a
4  # target element in a sorted list.
5  # • Instructions:
6  # o Prompt AI to create a function binary_search(arr,
7  # target) returning the index of the target or -1 if not
8  # found.
9  # o Include docstrings explaining best, average, and
10 # worst-case complexities.
11 # o Test with various inputs.
12 # • Expected Output:
13 # o Python code implementing binary search with AI-
14 # generated comments and docstrings
15 def binary_search(arr, target):
16     left, right = 0, len(arr) - 1
17
18     while left <= right:
19         mid = left + (right - left) // 2 # Calculate the middle index
20
21         # Check if the target is present at mid
22         if arr[mid] == target:
23             return mid
24         # If target is greater, ignore the left half
25         elif arr[mid] < target:
26             left = mid + 1
27         # If target is smaller, ignore the right half
28         else:
29             right = mid - 1
30
31     # Target was not found in the list
32     return -1
33 # Test cases
34 if __name__ == "__main__":
35     sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
36     targets = [5, 1, 10, 11]
37

```

Output:

```

ai_coding.py > ...
15 def binary_search(arr, target):
29     | | | | right = mid - 1
30
31     | | | # Target was not found in the list
32     | | | return -1
33
34     | | | # Test cases
35     | | | if __name__ == "__main__":
36     | | |     sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
37     | | |     targets = [5, 1, 10, 11]
38
39     | | |     for target in targets:
40     | | |         result = binary_search(sorted_list, target)
41     | | |         if result != -1:
42     | | |             print(f"Target {target} found at index: {result}")
43     | | |         else:
44     | | |             print(f"Target {target} not found in the list.")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDr
Target 5 found at index: 4
Target 1 found at index: 0
Target 10 found at index: 9
Target 11 not found in the list.
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

- Binary Search works on a sorted list by repeatedly checking the middle element and dividing the search range in half.
- If the target equals the middle → return index; if smaller → search left half; if larger → search right half.
- Time complexity is $O(\log n)$ (best case $O(1)$), making it much faster than linear search for large datasets.

Task Description #3 (Real-Time Application – Inventory Management System)

• **Scenario:** A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

• **Task:**

- o Use AI to suggest the most efficient search and sort algorithms for this use case.
- o Implement the recommended algorithms in Python.

- o Justify the choice based on dataset size, update frequency, and performance requirements.
- Expected Output:
 - o A table mapping operation → recommended algorithm → justification.
 - o Working Python functions for searching and sorting the inventory.

```

ai_coding.py 2 ...
1  # Task Description #3 (Real-Time Application - Inventory
2  # Management System)
3  # • Scenario: A retail store's inventory system contains
4  # thousands of products, each with attributes like product ID,
5  # name, price, and stock quantity. Store staff need to:
6  # 1. Quickly search for a product by ID or name.
7  # 2. Sort products by price or quantity for stock analysis.
8  # • Task:
9  # o Use AI to suggest the most efficient search and sort
10 # algorithms for this use case.
11 # o Implement the recommended algorithms in Python.
12 # o Justify the choice based on dataset size, update
13 # frequency, and performance requirements.
14 inventory = [
15     {"product_id": 1, "name": "Laptop", "price": 999.99, "quantity": 10},
16     {"product_id": 2, "name": "Smartphone", "price": 499.99, "quantity": 20},
17     {"product_id": 3, "name": "Headphones", "price": 199.99, "quantity": 15},
18     {"product_id": 4, "name": "Monitor", "price": 299.99, "quantity": 5},
19     {"product_id": 5, "name": "Keyboard", "price": 49.99, "quantity": 25},
20 ]
21 # Search function using Binary Search (for sorted data)
22 def binary_search(inventory, key, value):
23     inventory.sort(key=lambda x: x[key]) # Ensure the inventory is sorted
24     low, high = 0, len(inventory) - 1
25     while low <= high:
26         mid = (low + high) // 2
27         if inventory[mid][key] == value:
28             return inventory[mid]
29         elif inventory[mid][key] < value:
30             low = mid + 1
31         else:
32             high = mid - 1
33     return None
34 # Search function using Hash Table (for unsorted data)
35 def hash_table_search(inventory, key, value):
36     hash_table = {item[key]: item for item in inventory}
37     return hash_table.get(value, None)

```

```

ai_coding.py > ...
38 # Sort function using Quick Sort
39 def quick_sort(inventory, key):
40     if len(inventory) <= 1:
41         return inventory
42     pivot = inventory[len(inventory) // 2][key]
43     left = [x for x in inventory if x[key] < pivot]
44     middle = [x for x in inventory if x[key] == pivot]
45     right = [x for x in inventory if x[key] > pivot]
46     return quick_sort(left, key) + middle + quick_sort(right, key)
47 # Sort function using Merge Sort
48 def merge_sort(inventory, key):
49     if len(inventory) <= 1:
50         return inventory
51     mid = len(inventory) // 2
52     left = merge_sort(inventory[:mid], key)
53     right = merge_sort(inventory[mid:], key)
54     return merge(left, right, key)
55 def merge(left, right, key):
56     result = []
57     i = j = 0
58     while i < len(left) and j < len(right):
59         if left[i][key] <= right[j][key]:
60             result.append(left[i])
61             i += 1
62         else:
63             result.append(right[j])
64             j += 1
65     result.extend(left[i:])
66     result.extend(right[j:])
67     return result
68 # Example usage
69 # Searching for a product by name using Binary Search
70 product = binary_search(inventory, "name", "Laptop")
71 print("Binary Search Result:", product)
72 # Searching for a product by name using Hash Table
73 product = hash_table_search(inventory, "name", "Laptop")
74 print("Hash Table Search Result:", product)

```

Ln 13, Col 43 Spaces: 2 UTF-8 C

```

ai_coding.py > ...
74 print("Hash Table Search Result:", product)
75 # Sorting products by price using Quick Sort
76 sorted_inventory_quick = quick_sort(inventory, "price")
77 print("Sorted Inventory (Quick Sort):", sorted_inventory_quick)
78 # Sorting products by quantity using Merge Sort
79 sorted_inventory_merge = merge_sort(inventory, "quantity")
80 print("Sorted Inventory (Merge Sort):", sorted_inventory_merge)
81 # Operation → Recommended Algorithm → Justification
82 operation_algorithm_justification = [
83     {
84         "operation": "Search by ID or Name",
85         "recommended_algorithm": "Binary Search (for sorted data) or Hash Table (for unsorted data)",
86         "justification": "Binary Search is efficient for searching in sorted data with O(log n) time complexity, making it suitable for large dataset",
87     },
88     {
89         "operation": "Sort by Price or Quantity",
90         "recommended_algorithm": "Quick Sort (for large datasets) or Merge Sort (for stable sorting)",
91         "justification": "Quick Sort is efficient for large datasets with an average time complexity of O(n log n) and is generally faster than Merge Sort",
92     }
93 ]
94 # Print the operation → recommended algorithm → justification table
95 for item in operation_algorithm_justification:
96     print(f"Operation: {item['operation']}")
97     print(f"Recommended Algorithm: {item['recommended_algorithm']}")
98     print(f"Justification: {item['justification']}\n")
99
100

```

OUTPUT:

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:\Users\Eshwar\OneDrive\Desktop\python\ai_coding.py
Binary Search Result: {'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'quantity': 10}
Hash Table Search Result: {'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'quantity': 10}
Sorted Inventory (Quick Sort): [{'product_id': 5, 'name': 'Keyboard', 'price': 49.99, 'quantity': 25}, {'product_id': 3, 'name': 'Headphones', 'price': 199.99, 'quantity': 15}, {'product_id': 4, 'name': 'Monitor', 'price': 299.99, 'quantity': 5}, {'product_id': 2, 'name': 'Smartphone', 'price': 499.99, 'quantity': 20}, {'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'quantity': 10}]
Sorted Inventory (Merge Sort): [{'product_id': 4, 'name': 'Monitor', 'price': 299.99, 'quantity': 5}, {'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'quantity': 10}, {'product_id': 3, 'name': 'Headphones', 'price': 199.99, 'quantity': 15}, {'product_id': 2, 'name': 'Smartphone', 'price': 499.99, 'quantity': 20}, {'product_id': 5, 'name': 'Keyboard', 'price': 49.99, 'quantity': 25}]
Operation: Search by ID or Name
Recommended Algorithm: Binary Search (for sorted data) or Hash Table (for unsorted data)
Justification: Binary Search is efficient for searching in sorted data with O(log n) time complexity, making it suitable for large datasets. However, if the data is frequently updated, a Hash Table can provide O(1) average time complexity for search operations.

```

Explanation:

- Use a dictionary (hash table) to search products by ID quickly in $O(1)$ time and linear search for names in $O(n)$.
- Use Python's `sorted()` (Timsort) to sort products by price or quantity efficiently in $O(n \log n)$.
- This combination is best for large, frequently updated inventories because it provides fast lookup and efficient sorting.

ask description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms in terms of efficiency and suitability.
- Implement the recommended algorithms in Python.


```

ai_coding.py > ...
1  # ask description #4: Smart Hospital Patient Management
2  # System
3  # A hospital maintains records of thousands of patients with details
4  # such as patient ID, name, severity level, admission date, and bill
5  # amount. Doctors and staff need to:
6  # 1. Quickly search patient records using patient ID or name.
7  # 2. Sort patients based on severity level or bill amount for
8  # prioritization and billing.
9  # Student Task
10 # • Use AI to recommend suitable searching and sorting
11 # algorithms.
12 # • Justify the selected algorithms in terms of efficiency and
13 # suitability.
14 # • Implement the recommended algorithms in Python.
15 class Patient:
16     def __init__(self, patient_id, name, severity_level, admission_date, bill_amount):
17         self.patient_id = patient_id
18         self.name = name
19         self.severity_level = severity_level
20         self.admission_date = admission_date
21         self.bill_amount = bill_amount
22
23 class Hospital:
24     def __init__(self):
25         self.patients_by_id = {}
26         self.patients_by_name = {}
27
28     def add_patient(self, patient):
29         self.patients_by_id[patient.patient_id] = patient
30         self.patients_by_name[patient.name] = patient.patient_id
31
32     def search_by_id(self, patient_id):
33         return self.patients_by_id.get(patient_id, None)
34
35     def search_by_name(self, name):
36         patient_id = self.patients_by_name.get(name, None)
37         if patient_id:

```

Ln 14, Col 52 Spaces: 2

```

ai_coding.py > ...
23 class Hospital:
35     def search_by_name(self, name):
38         return self.patients_by_id[patient_id]
39         return None
40
41     def sort_patients(self, key):
42         if key == 'severity':
43             return sorted(self.patients_by_id.values(), key=lambda x: x.severity_level, reverse=True)
44         elif key == 'bill':
45             return sorted(self.patients_by_id.values(), key=lambda x: x.bill_amount, reverse=True)
46         else:
47             raise ValueError("Invalid sorting key. Use 'severity' or 'bill'.")
48 # Example Usage
49 hospital = Hospital()
50 hospital.add_patient(Patient(1, "John Doe", 5, "2024-01-01", 1000))
51 hospital.add_patient(Patient(2, "Jane Smith", 3, "2024-01-02", 500))
52 hospital.add_patient(Patient(3, "Alice Johnson", 4, "2024-01-03", 1500))
53 # Search by patient ID
54 patient = hospital.search_by_id(1)
55 print(patient.name) # Output: John Doe
56 # Search by name
57 patient = hospital.search_by_name("Jane Smith")
58 print(patient.patient_id) # Output: 2
59 # Sort by severity level
60 sorted_patients = hospital.sort_patients('severity')
61 for patient in sorted_patients:
62     print(patient.name, patient.severity_level) # Output: John Doe 5, Alice Johnson 4, Jane Smith 3
63 # Sort by bill amount
64 sorted_patients = hospital.sort_patients('bill')
65 for patient in sorted_patients:
66     print(patient.name, patient.bill_amount) # Output: Alice Johnson 1500, John Doe 1000, Jane Smith 500
67

```

Output:

```
PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/a
John Doe
2
John Doe 5
Alice Johnson 4
Jane Smith 3
Alice Johnson 1500
John Doe 1000
Jane Smith 500
PS C:\Users\Eshwar\OneDrive\Desktop\python> |
```

Explanation:

- Use a dictionary (hash table) to search patients by ID in $O(1)$ time and linear search for names in $O(n)$.
- Use Python's `sorted()` (Timsort) to sort patients by severity or bill efficiently in $O(n \log n)$.
- This approach is ideal because it ensures fast lookup and efficient sorting for large hospital datasets.

Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.
2. Sort students based on marks to generate rank lists.

Student Task

- Identify efficient searching and sorting algorithms using AI assistance.
- Justify the choice of algorithms.
- Implement the algorithms in Python.

```

ai_coding.py > binary_search
1  # Task Description #5: University Examination Result Processing
2  # System
3  # A university processes examination results for thousands of students
4  # containing roll number, name, subject, and marks. The system must:
5  # 1. Search student results using roll number.
6  # 2. Sort students based on marks to generate rank lists.
7  # Student Task
8  # • Identify efficient searching and sorting algorithms using AI
9  # assistance.
10 # • Justify the choice of algorithms.
11 # • Implement the algorithms in Python.
12 # Efficient Searching Algorithm: Binary Search
13 # Justification: Binary search is efficient for searching in sorted lists, with a time complexity of  $O(\log n)$ . Since the student re
14 # Efficient Sorting Algorithm: Merge Sort
15 # Justification: Merge sort is a stable sorting algorithm with a time complexity of  $O(n \log n)$  in all cases. It is efficient for so
16 # Implementation in Python
17 class Student:
18     def __init__(self, roll_number, name, subject, marks):
19         self.roll_number = roll_number
20         self.name = name
21         self.subject = subject
22         self.marks = marks
23
24 def binary_search(students, roll_number):
25     left, right = 0, len(students) - 1
26     while left <= right:
27         mid = left + (right - left) // 2
28         if students[mid].roll_number == roll_number:
29             return students[mid]
30         elif students[mid].roll_number < roll_number:
31             left = mid + 1
32         else:
33             right = mid - 1
34     return None
35 def merge_sort(students):
36     if len(students) > 1:
37         mid = len(students) // 2

```

```
al_coding.py / merge_sort
35 def merge_sort(students):
38     left_half = students[:mid]
39     right_half = students[mid:]
40
41     merge_sort(left_half)
42     merge_sort(right_half)
43
44     i = j = k = 0
45     while i < len(left_half) and j < len(right_half):
46         if left_half[i].marks > right_half[j].marks:
47             students[k] = left_half[i]
48             i += 1
49         else:
50             students[k] = right_half[j]
51             j += 1
52         k += 1
53
54     while i < len(left_half):
55         students[k] = left_half[i]
56         i += 1
57         k += 1
58
59     while j < len(right_half):
60         students[k] = right_half[j]
61         j += 1
62         k += 1
63 # Example usage
64 students = [
65     Student(101, "Alice", "Math", 85),
66     Student(102, "Bob", "Math", 90),
67     Student(103, "Charlie", "Math", 80)
68 ]
69 # Sort students by marks
70 merge_sort(students)
71 # Search for a student by roll number
72 result = binary_search(students, 102)
73 if result:
```

Output:

```

ai_coding.py > merge_sort
67 | | Student(103, "Charlie", "Math", 80)
68 | ]
69 | # Sort students by marks
70 | merge_sort(students)
71 | # Search for a student by roll number
72 | result = binary_search(students, 102)
73 | if result:
74 | | print(f"Student found: {result.name} with marks {result.marks}")
75 | else:
76 | | print("Student not found.")
77 |
78 |

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/pytho
Student not found.
PS C:\Users\Eshwar\OneDrive\Desktop\python>

```

Explanation:

- Use a dictionary (hash table) to store student records by roll number so searching is very fast — $O(1)$ time.
- Use Python's built-in sorted() (Timsort) to sort students by marks for ranking — $O(n \log n)$.
- This approach is efficient and scalable because it combines instant lookup with fast sorting for large datasets.

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the algorithm selection.
- Implement searching and sorting modules in Python

```

ai_coding.py > ...
1 # Task Description #6: Online Food Delivery Platform
2 # An online food delivery application stores thousands of orders with
3 # order ID, restaurant name, delivery time, price, and order status. The
4 # platform needs to:
5 # 1. Quickly find an order using order ID.
6 # 2. Sort orders based on delivery time or price.
7 # Student Task
8 # • Use AI to suggest optimized algorithms.
9 # • Justify the algorithm selection.
10 # • Implement searching and sorting modules in Python
11 # Optimized Algorithms:
12 # 1. For quickly finding an order using order ID, we can use a hash table
13 # (dictionary in Python) to store orders. This allows for O(1) average time complexity for lookups.
14 # 2. For sorting orders based on delivery time or price, we can use the built
15 # in `sorted()` function in Python, which implements Timsort (a hybrid sorting algorithm derived from merge sort and insertion sort). Timsort has
16
17 # Implementation of searching and sorting modules in Python
18 class Order:
19     def __init__(self, order_id, restaurant_name, delivery_time, price, order_status):
20         self.order_id = order_id
21         self.restaurant_name = restaurant_name
22         self.delivery_time = delivery_time
23         self.price = price
24         self.order_status = order_status
25
26     def __repr__(self):
27         return f"Order({self.order_id}, {self.restaurant_name}, {self.delivery_time}, {self.price}, {self.order_status})"
28
29 class FoodDeliveryPlatform:
30     def __init__(self):
31         self.orders = {}
32
33     def add_order(self, order):
34         self.orders[order.order_id] = order
35
36     def find_order_by_id(self, order_id):
37         return self.orders.get(order_id, None)

```

Output:

```

ai_coding.py > ...
29 class FoodDeliveryPlatform:
30     def find_order_by_id(self, order_id):
31         return self.orders.get(order_id, None)
32
33     def sort_orders_by_delivery_time(self):
34         return sorted(self.orders.values(), key=lambda x: x.delivery_time)
35
36     def sort_orders_by_price(self):
37         return sorted(self.orders.values(), key=lambda x: x.price)
38
39 # Example usage
40 platform = FoodDeliveryPlatform()
41 platform.add_order(Order(1, "Pizza Place", "2024-06-01 18:00", 20.0, "Delivered"))
42 platform.add_order(Order(2, "Sushi Spot", "2024-06-01 19:00", 35.0, "Pending"))
43 platform.add_order(Order(3, "Burger Joint", "2024-06-01 17:30", 15.0, "Delivered"))
44 print(platform.find_order_by_id(2)) # Output: Order(2, Sushi Spot, 2024-06-01 19:00, 35.0, Pending)
45 print(platform.sort_orders_by_delivery_time()) # Output: [Order(3, Burger Joint
46
47
48
49
50
51
52

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Eshwar\OneDrive\Desktop\python> & "C:\Program Files\Python314\python.exe" c:\Users\Eshwar\OneDrive\Desktop\python\ai_coding.py
Order(2, Sushi Spot, 2024-06-01 19:00, 35.0, Pending)
[Order(3, Burger Joint, 2024-06-01 17:30, 15.0, Delivered), Order(1, Pizza Place, 2024-06-01 18:00, 20.0, Delivered), Order(2, Sushi Spot, 2024-06-01 19:00, 35.0, Pending)]
PS C:\Users\Eshwar\OneDrive\Desktop\python>

Explanation:

- Use a dictionary (hash table) to store orders by ID so searching is very fast — $O(1)$ time complexity.
- Use Python's built-in `sorted()` (Timsort) to sort orders by price or delivery time efficiently — $O(n \log n)$.
- This combination is optimal because it provides instant search + fast sorting for large datasets.

